ML Model Deployment Strategies and Model Serving using FastAPI

CLA-4 Assignment Report

1. Project Overview

This project uses the Titanic – Machine Learning from Disaster dataset from Kaggle to train a classification model and demonstrate three inference strategies: Online Inference, Batch Inference, and Streaming Inference. The same saved machine learning pipeline is also served through a REST API using FastAPI.

2. Dataset

Dataset	Titanic – Machine Learning from Disaster
Source	Kaggle
File used	train.csv
Dataset link	https://www.kaggle.com/competitions/titanic/data

3. Model Training and Results

The labelled dataset was split into training and testing portions. Necessary preprocessing was applied and a classification model was trained. The complete trained pipeline was saved using Joblib as **model.pkl** so that the same model could be reused for all inference strategies and FastAPI serving.

Result	Output
Model training	Completed successfully
Test accuracy	81.56%
Saved model	model.pkl
Batch inference	20 unseen records processed
Streaming inference	10 unseen records processed one at a time
FastAPI /health	HTTP 200 – API is running
FastAPI /predict	HTTP 200 – prediction returned
Invalid request	HTTP 422 – validation error
Python client	HTTP 200 – prediction returned

4. Observations

1. Online Inference vs Batch Inference

Online Inference processes one record or request at a time and returns a prediction immediately. Batch Inference processes multiple records together in a single operation. Therefore, online inference is suitable when an immediate response is required, while batch inference is suitable for processing a larger group of records efficiently.

2. Streaming Inference vs Batch Inference

Streaming Inference processes records continuously as they arrive, one record at a time, with low latency. Batch Inference collects a group of records and processes them together. In this assignment, streaming was simulated using 10 unseen records with a 1-second delay between records.

3. Why FastAPI is used for Model Serving

FastAPI is used to expose the trained machine learning model through REST API endpoints. It provides simple and fast request handling, Pydantic-based input validation, and automatic interactive API documentation through Swagger UI. This makes it convenient to test and integrate the model with other applications.

4. Why the same trained model is reused

The same saved trained pipeline should be reused so that all inference strategies use identical preprocessing and model logic. This provides consistent predictions, avoids unnecessary retraining, and ensures that the deployment results correspond to the evaluated trained model.

5. Conclusion

The assignment successfully demonstrates the complete ML deployment workflow: training and saving a model, performing online, batch and streaming inference, and serving the same trained model through FastAPI. The API was validated using both valid and invalid requests, and the prediction endpoint was also tested using a Python client.

6. Implementation Files

The project includes the training program, online/batch/streaming inference programs, FastAPI application, Python client, saved model, batch prediction CSV, requirements file, README, and screenshots of the completed tests and outputs.